CROSS-TEAM COLLABORATION

Team Design and Dependency Discovery

Conway's Law as the foundational premise for team design. Principles for drawing good team boundaries: aligning to value streams, matching size to communication capacity, separating concerns by rate of change, and making ownership explicit. End-to-end, capability, and enabling team types and their trade-offs. How to surface dependencies between teams, read the patterns they reveal, and choose the right structural response.

What this resource is for

How an organisation structures its teams determines, more than almost anything else, how information flows, how decisions get made, and how work reaches delivery. Teams that are structured poorly generate coordination cost, communication overhead, and delivery bottlenecks that no amount of process improvement can fully resolve. In AI-accelerated environments, where the tempo of output has increased but organisational structures were often designed for a slower pace, the mismatch between team design and delivery demand is one of the most significant — and most fixable — sources of friction.

This resource covers two connected topics: how to think about team design as a deliberate act of organisational choice, and how to surface the dependencies between teams that reveal where the current structure is creating unnecessary cost.

Structure shapes communication and communication shapes the product

The most important starting point for team design is Conway's Law, an observation made by computer scientist Melvin Conway in 1968: organisations tend to produce systems that mirror their own communication structures. If three teams are building a product together, the product will tend to develop three major components — not necessarily because that is the right architecture, but because each team builds what it can build independently and integrates at the points where it must communicate with others.

Conway's Law is not a technical observation — it is an organisational one, and it applies far beyond software. The implication is significant in both directions. If you want to change how a system or product is structured, you often need to change how the teams building it are organised. And if you reorganise teams without thinking about the work they are trying to coordinate, the new structure will shape the work in ways you did not intend.

The practical lesson is that team design is not primarily an HR or resource question — it is a product and delivery question. The boundaries you draw around teams determine what those teams can do independently and what they will need to negotiate with others. Drawing those boundaries well is one of the highest-leverage interventions available to a leader in a complex organisation.

What makes a team boundary good or bad

A team boundary is the line that defines what a team owns and is accountable for. Good team boundaries share a few properties.

A good boundary encloses a coherent domain of work. The things inside the boundary belong together — they change together, they are understood together, and they are best decided on together. When a boundary cuts across a naturally coherent domain, the two teams on either side will need to coordinate constantly on things that should be internal decisions. This coordination is not free — it consumes time, introduces latency, and creates opportunities for misalignment.

A good boundary minimises mandatory coordination. Every time one team needs something from another team in order to proceed, a dependency exists. Dependencies are not inherently bad, but every dependency is a potential bottleneck. A team that can complete most of its important work without waiting for other teams will deliver faster and more predictably than one that is constantly blocked on others. The goal of boundary design is to make the most frequent and most critical work as independent as possible, pushing coordination needs toward the edges.

A good boundary comes with clear ownership. The team on the inside of the boundary is accountable for what happens there. They make the decisions, they bear the consequences, and they do not need to seek approval from outside for routine choices within their domain. Boundary ambiguity — areas where two teams both believe, or neither believes, they are responsible — is one of the most common sources of both conflict and delivery failure.

Principles for designing team structures

Several principles from organisational design research are durable and well-supported, regardless of the specific framework you apply them through.

Align teams to value streams, not functions. A value stream is the sequence of activities required to deliver something of value to a user or customer. When teams are aligned to value streams — owning an end-to-end capability — they can make decisions and deliver independently. When teams are aligned to functions — one team for design, one for engineering, one for testing — every feature requires multiple teams to coordinate, and the organisational structure itself becomes a bottleneck. Functional structures are good at building deep expertise; they are poor at delivering integrated value quickly.

Match team size to communication capacity. There is a well-established relationship between team size and communication overhead. As team size grows, the number of communication channels grows faster — a team of five has ten possible communication pairs; a team of ten has forty-five. This is not just a

theoretical concern: empirically, small teams communicate more effectively, make faster decisions, and maintain shared context more reliably than large ones. The commonly cited range of five to nine people for a working team is grounded in this reality, not arbitrary convention.

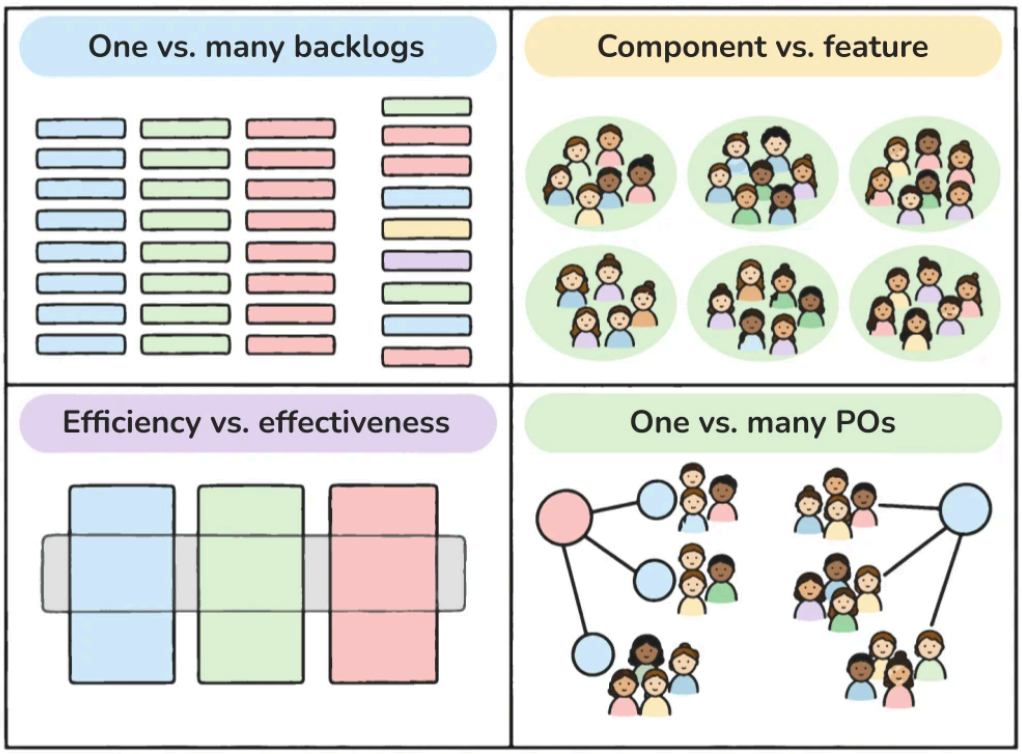
Separate concerns at different rates of change. Some parts of a system or product change frequently; others change rarely. When fast-changing and slow-changing concerns are owned by the same team, the team's rhythm is determined by the fastest-changing element — which may not be appropriate for everything it owns. Separating concerns by rate of change allows each team to operate at a pace that matches its domain.

Make ownership explicit and complete. Every significant capability, system component, or customer-facing outcome should have a clear owning team. Gaps in ownership — things that everyone assumes someone else is responsible for — reliably produce delivery failures and escalating conflicts.

A complete ownership map, even a rough one, is more valuable than a sophisticated organisational chart that leaves accountability implicit.

Team types and their trade-offs

While no single framework has a monopoly on how to categorise teams, a few recurring distinctions are worth understanding because they reflect real structural trade-offs.



End-to-end teams own a complete user journey or customer capability from interface to infrastructure. They are responsible for outcomes, not just outputs — for whether the user can successfully do something, not just for whether a component was built. End-to-end ownership maximises accountability and minimises handoffs, but it requires breadth of skill within the team and can create tension when a technical component is shared across multiple journeys.

Capability or platform teams provide shared services or infrastructure that other teams consume. Rather than building every team's own solution to a common problem, a capability team builds it once and makes it available. This pattern is efficient when the shared capability is genuinely common and stable — but it creates a dependency for every team that relies on it. A slow, unreliable, or unresponsive capability team becomes a bottleneck for everyone downstream.

Enabling or coaching teams are temporary structures whose purpose is to build capability in other teams — not to do work on their behalf. An enabling team works alongside other teams for a defined period, transfers knowledge and practice, and then moves on. The measure of their success is whether the teams they worked with no longer need them.

The most important question when choosing a team type is not which pattern is theoretically best but which best minimises the coordination cost for the most important work in your specific context. There is no universally correct answer, and the right structure for a fifty-person organisation is almost certainly wrong for a five-hundred-person one.

Dependency discovery

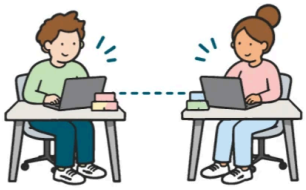
Dependencies between teams are one of the most significant and most underexamined sources of delivery slowdown. A dependency exists whenever one team cannot complete meaningful work without something from another team. Dependencies are not inherently bad — in any organisation where teams specialise, some dependencies are inevitable. But unexamined dependencies create invisible bottlenecks, unpredictable delivery timelines, and recurring friction between teams that each believe the other is the problem.

Dependency discovery is the practice of making these dependencies visible before they become crises. It is most valuable at the start of a delivery cycle or planning period, when there is still time to act on what is found.



Hard dependency

Work can't continue without input from another team.



Soft dependency

Work is slower or lower quality without coordination.



Shared resource

Several teams rely on the same person, tool or platform.

Levers to loosen dependencies

- Add a skill to the team
- Delegate the decision
- Create a shared service
- Change the interface

What to look for. Dependencies fall into a few recurring categories. Technical dependencies arise when one team's system or output is an input to another team's work. Knowledge dependencies arise when one team needs expertise or context that lives in another team. Approval dependencies arise when one team requires sign-off from another team or function before proceeding. Resource dependencies arise when teams share access to people, infrastructure, or tooling that is finite. Dependencies also differ in how tightly they bind. With a hard dependency, work cannot continue without input from another team. With a soft dependency, work is slower or of lower quality without coordination. A shared resource is a person, tool, or platform that several teams rely on.

How to surface them. The most reliable method is a structured conversation, not a survey. Bring team representatives together and ask, for each significant piece of planned work: what do we need from other teams that we cannot provide ourselves? What other teams need something from us that would block them if we were delayed? These questions surface both outgoing and incoming dependencies — both matter.

For each dependency identified, note: which teams are involved; what specifically is needed; how frequently this dependency arises; how critical it is (does it block delivery, or just slow it?); and whether there is currently a clear protocol for managing it.

Reading the map. Once dependencies are visible, patterns become apparent. Concentration is the most important pattern to look for: if one team or component appears as a dependency for many other teams, it is a structural bottleneck. Any delay, instability, or capacity constraint in that team propagates across everyone who depends on it. Circular dependencies — where Team A needs something from Team B, and Team B needs something back from Team A — are a second critical pattern. Circular dependencies can create mutual blockages that neither team can resolve independently.

Acting on what you find. There are three structural responses to a problematic dependency. You can loosen it by building the needed capability into the dependent team, so they no longer need to wait. You can formalise it by putting a clear interface, agreement, or service commitment in place — converting an ad-hoc coordination need into a predictable relationship. Or you can eliminate it by redesigning team ownership so that the things that need to go together are owned by the same team. In practice this comes down to four levers: add a skill to the team, delegate the decision, create a shared service, or change the interface. The right response depends on the nature of the dependency, the frequency and criticality of the need, and what is feasible given the current structure and skills.

The connection to coordination patterns

Dependency discovery is not an end in itself — it is the input to coordination design. Once you know what your dependencies are, the question becomes: what is the right mechanism for managing each one? That question is covered in Coordination Patterns and Flight Levels, which maps specific coordination approaches to specific dependency types and organisational contexts.

Further reading

- Conway, M. — "How Do Committees Invent?" (Datamation, 1968) — the original statement of Conway's Law, brief and still worth reading
- Galbraith, J.R. — Designing Organizations: Strategy, Structure, and Process at the Business Unit and Enterprise Levels — the foundational organisational design reference for information processing and coordination
- Hackman, J.R. — Leading Teams: Setting the Stage for Great Performances — covers team boundaries, authority, and the structural conditions that determine team effectiveness
- Reinertsen, D. — The Principles of Product Development Flow — applies queuing theory and systems thinking to dependency cost and coordination overhead in knowledge work
- Forsgren, N., Humble, J., & Kim, G. — Accelerate: The Science of Lean Software and DevOps — empirical evidence for which organisational and structural conditions predict delivery performance

Online version with links to related resources: https://resources.simsab.net/coordination-collaboration-ai/team-design-and-dependency-discovery